

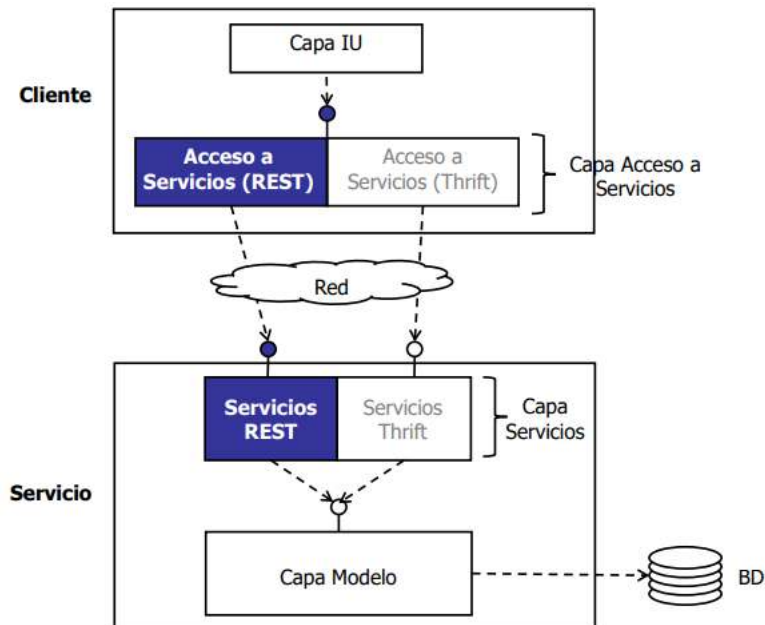
TEMA 6: DISEÑO E IMPLEMENTACIÓN DE SERVICIOS WEB REST

DESCRIPCIÓN DEL CASO DE ESTUDIO

DESCRIPCIÓN DEL CASO DE ESTUDIO

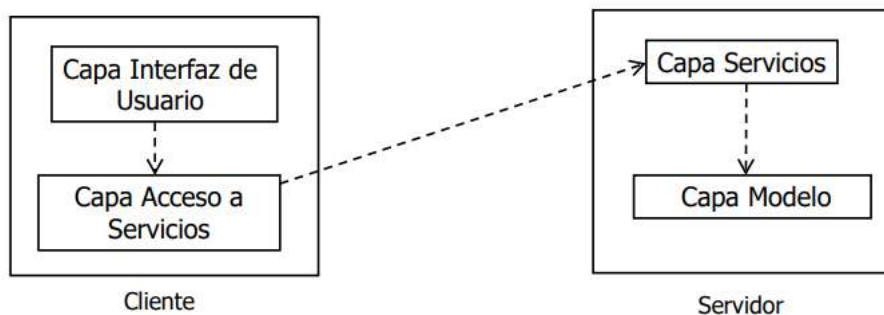
- Ejemplo ilustrado en los temas 3 y 5
- Se expondrá la capa modelo desarrollada como un servicio web usando REST y como un servicio Apache Thrift
- En el ejemplo, el cliente será una sencilla aplicación de línea de comandos
- El cliente puede configurarse (sin necesidad de recompilación) para usar el servicio web REST o Thrift

ARQUITECTURA GENERAL DE MOVIES



DISEÑO POR CAPAS

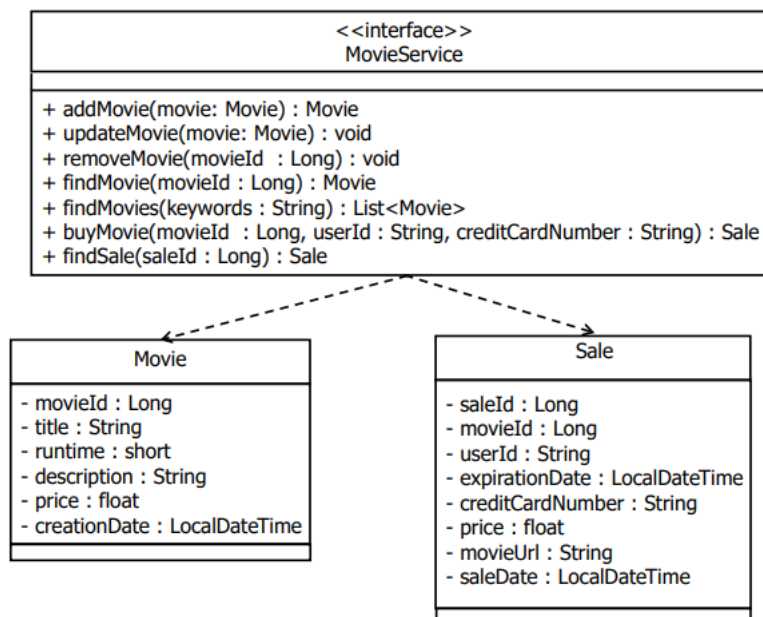
- Se usará el diseño por capas para ocultar tecnologías de acceso e implementación de los servicios



- **Capa Interfaz de Usuario (o lógica de la aplicación cliente)**
 - Implementa la interfaz de usuario (y/o la lógica de la aplicación cliente)
 - No depende de la tecnología de acceso al servicio
- **Capa Acceso a Servicios**
 - Implementa el acceso a los servicios
 - Ofrece una API que oculta la tecnología usada para acceder a los servicios
- **Capa Servicios**
 - Utiliza una tecnología para implementar los servicios
 - Delega en la capa Modelo
- **Capa Modelo**
 - Implementa la lógica de la aplicación
 - No depende de la tecnología de implementación de los servicios

- El uso del diseño por capas en este caso tiene las ventajas y riesgos habituales
 - La persona/s que se encarga del desarrollo de las capas IU y Modelo no tienen por qué saber nada sobre las tecnologías de los servicios
 - Cada capa puede ser desarrollada en paralelo
 - Se facilita el mantenimiento del software
 - Cambios en la implementación de una capa (ej. uso de nuevas tecnologías) no conllevan cambios en el resto de las capas
- Riesgos
 - Software más complejo
 - Si el software a construir es muy sencillo (ej. prototipo) puede no valer la pena
 - A veces, que el modelo conozca los detalles de la tecnología de implementación del servicio podría permitir ciertas optimizaciones

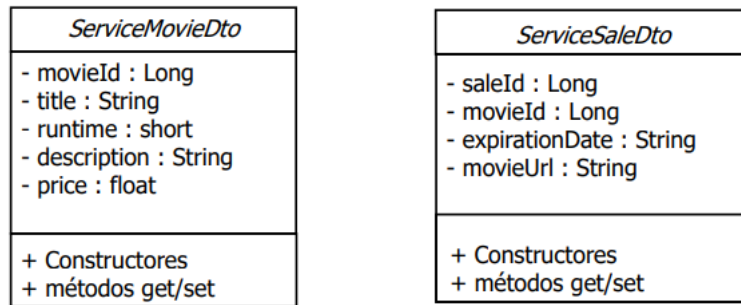
INTERFAZ DE LA CAPA MODELO



CAPA SERVICIOS

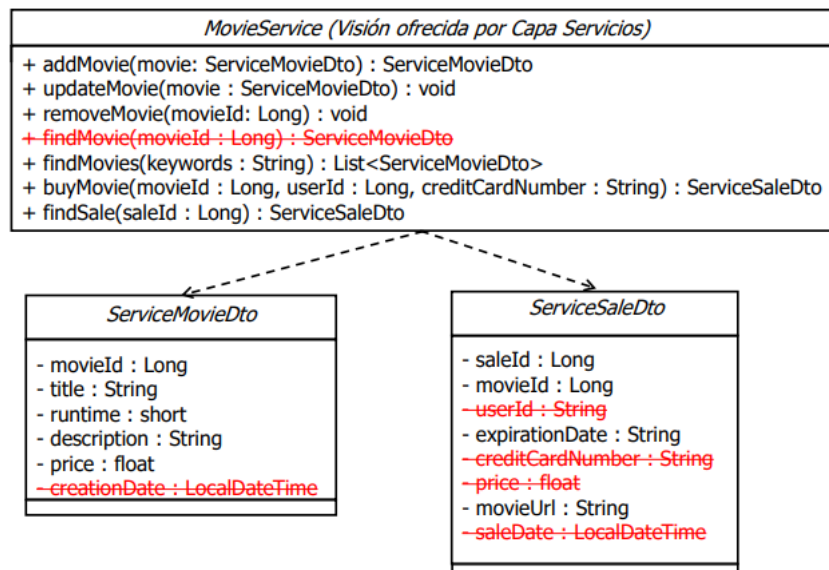
- Expone remotamente la capa modelo creada en el tema 3, pero los servicios no exponen directamente los objetos **Movie** y **Sale** del modelo
- Este ejemplo asume que el acceso remoto lo van a realizar aplicaciones externas
 - La fecha de creación de la película en la BD es un dato interno que no se desea exponer a aplicaciones externas
 - En las ventas, los clientes externos sólo necesitan los identificadores de venta y película, la fecha de expiración y la url para ver la película
- Los servicios utilizarán 2 objetos alternativos → **ServiceMovieDto** y **ServiceSaleDto**
- DTO (Data Transfer Object) → objeto para transferir datos entre aplicaciones → encapsula diferencias con los objetos internos de cada aplicación según
 - Relevancia → no son relevantes externamente (caso de los ejemplos)
 - Grosor → se quieren objetos más “gruesos” para minimizar llamadas remotas
 - Ej
 - Se supone que también se tiene información de actores
 - Los actores tienen su propia tabla en la BD, su clase y su DAO
 - Entre otros casos de uso para manejar información de actores, el modelo ofrece el método **getActorsByMovie**, que recibe un **movieId** y devuelve una lista de actores
 - Si se sabe que la mayoría de los clientes, al buscar información de películas van a querer buscar los actores, se podría hacer que el objeto **ServiceMovieDto** incluyese ya la lista
 - La operación **findMovies** del servicio invocaría a **findMovies** del Modelo y, para cada película, invocaría a **getActorsByMovie** para obtener los actores

- Visión ofrecida por la capa servicios

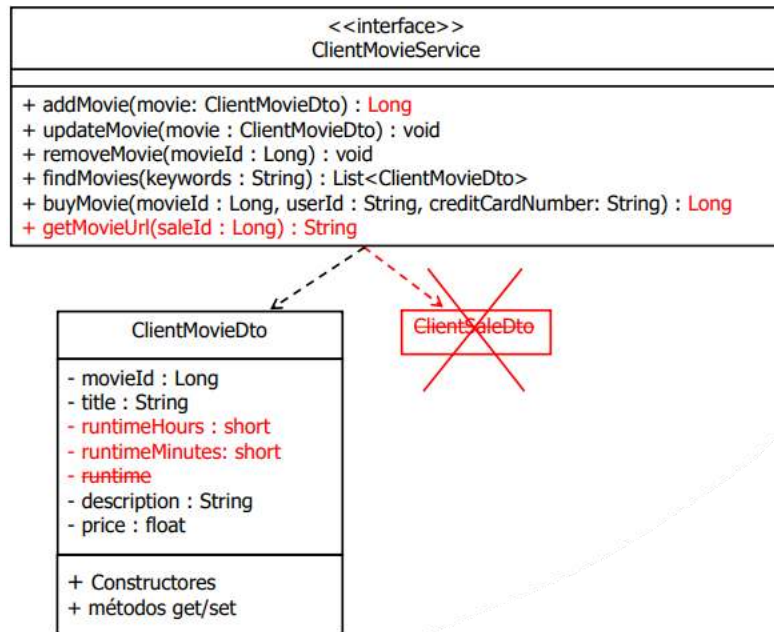


- A las aplicaciones externas se les expone una API diferente que a las internas
 - Diferencias en los objetos expuestos
 - No se expone la operación **findMovie**
 - En casos más complejos podrían ser necesarias validaciones o pasos adicionales antes de invocara a la capa modelo
- En el ejemplo, se asume que las aplicaciones internas que necesiten acceder a más datos de las películas y ventas que los expuestos por la capa servicios diseñada para la aplicaciones externas, accederían a través de otra capa Servicios con una API menos restringida
- **NOTA** → en **ServiceSaleDto** la propiedad **expirationDate** es tipo **String** en lugar de **LocalDateTime**
 - La fecha se envía al cliente como un String

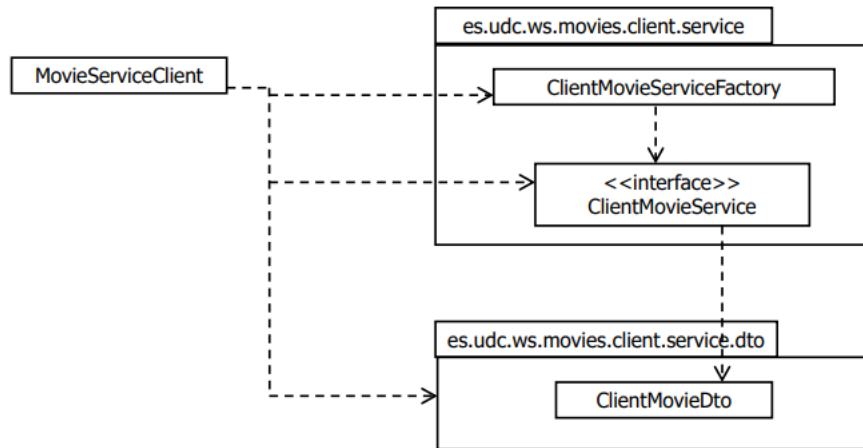
INTERFAZ DE LA CAPA SERVICIOS



INTERFAZ DE LA CAPA ACCESO A SERVICIOS



CAPA INTERFAZ DE USUARIO



- Cliente **MovieServiceClient**
 - Añadir película
 - **MovieServiceClient -a <title> <hours> <minutes> <description> <price>**
 - Borrar película
 - **MovieServiceClient -r <movieId>**
 - Actualizar película
 - **MovieServiceClient -u <movieId> <title> <hours> <minutes> <description> <prince>**
 - Buscar películas por keywords en el título
 - **MovieServiceClient -f <keywords>**
 - Comprar película
 - **MovieServiceClient -b <movieId> <userId> <creditCardNumber>**
 - Ver película (recuperar venta)
 - **MovieServiceClient -g <saleId>**
- NOTA
 - Se tiene un único cliente que puede ejecutarse desde la intranet del servicio o desde Internet, pero en un caso real, posiblemente habría al menos 2 tipos de clientes
 - Uno para las labores de administración que se ejecutaría normalmente desde la intranet del servicio
 - Otro para buscar, comprar y ver películas que se usaría normalmente desde Internet
 - El cliente especifica la duración de la película separándola en horas y minutos

INTRODUCCIÓN A LOS SERVICIOS WEB REST

INTRODUCCIÓN

- REpresentational State Transfer → estilo arquitectónico propuesto por Roy Fielding en 2000
- La Web → la aplicación distribuida más exitosa de la historia
- REST → estilo arquitectónico para construir aplicaciones distribuidas inspirado en las características de la Web
- Independiente de la tecnología, pero se suele implementar usando directamente HTTP y algún lenguaje de intercambio (XML o JSON), sin tecnologías adicionales
 - A menudo los servicios REST reales no siguen estrictamente todos los principios del estilo arquitectónico REST
 - A los servicios que sí siguen todos los principios REST se les denomina servicios web RESTful

RECORDATORIO DE HTTP

- HTTP → HyperText Transfer Protocol
 - Estandarizado por el W3C y la IETF
- HTTP 3 es la última versión
- Protocolo cliente/servidor utilizado en la Web
- Inicialmente construido para transferir páginas HTML, pero puede transferir cualquier contenido
 - Las peticiones y respuestas HTTP usan las cabeceras Accept y Content-Type para indicar el tipo de MIME (ej. text/html, application/json, image/jpeg, etc.) que indica la naturaleza de la información que se espera recibir (ej. Accept: application/json) o se envía en el cuerpo (ej. Content-Type: application/json)
- Esquema petición/respuesta

PETICIONES HTTP

- Concepto clave → URL como identificador **global** de recursos
- Una petición HTTP consta de
 - URL → identifica al recurso sobre el que actúa la petición
 - Método de acceso (GET, POST, PUT, ...), que especifica la acción a realizar sobre el recurso
 - Cabeceras
 - Metainformación de la petición que indican información adicional que puede ser necesaria para procesar la petición (ej. información de autenticación)
 - Cuerpo del mensaje (opcional)
 - Solo con algunos métodos de acceso
- Métodos de acceso
 - GET
 - Solicita una representación del recurso especificado (ej. página HTML)
 - Seguro (*safe*)
 - Semántica esencialmente de solo lectura
 - El cliente no pide ni espera que se produzcan cambios en el servidor como resultado de realizar una petición sobre un recurso utilizando este método
 - Un cliente puede utilizarlo sin miedo a causar efectos indeseados
 - Pueden especificarse parámetros en la URL (consultas)
 - Se especifican como pares campo=valor, separadas por el carácter '&'
 - Ej. <http://www.bookshop.com/search?tit=Java&author=John+Smith>
 - PUT
 - Carga en el servidor una representación de un recurso (que podría existir previamente o no)
 - La nueva representación del recurso va en el cuerpo de la petición
 - No seguro
 - DELETE
 - Elimina el recurso especificado
 - No seguro

- POST
 - Envía datos al recurso indicado para que los procese
 - Los datos van en el cuerpo de la petición
 - Puede utilizarse para solicitar la ejecución de cualquier acción en el servidor
 - No seguro
- Peticiones GET, PUT, DELETE deben ser idempotentes
 - Múltiples peticiones iguales deben tener el mismo efecto en el servidor que una sola
- Las peticiones POST no tienen porque ser idempotentes
- Cabeceras de la petición
 - Tipo MIME de datos enviados en el cuerpo
 - Tipo MIME de datos esperados (ej. HTML, XML, JSON, ...)
 - El cliente puede especificar qué formatos entiende y en qué formato prefiere recibir la representación
 - Encoding esperado
 - Lenguaje esperado
 - Peticiones condicionales (If-Modified-Since, Etags)
 - Solicita una representación sólo si ha cambiado desde un momento determinado
 - Caches en el cliente
 - Credenciales de autenticación / autorización
 - Información para proxies
 - Agente del usuario (ej. navegador utilizado)
 - etc.

REPUESTA HTTP

- Contenido respuesta HTTP
 - Código de Status
 - 200 OK
 - 201 Created
 - 400 Bad Request
 - 403 Forbidden
 - 404 Not Found
 - 500 Internal Error
 - Lista completa → <https://datatracker.ietf.org/doc/html/rfc7231#section-6>
 - Cabeceras → metainformación de la respuesta
 - Cuerpo del mensaje (opcional)
 - GET → representación del recurso invocado
 - POST → vacío o con el resultado del procesamiento realizado por el servidor
 - PUT → normalmente, el cuerpo viene vacío
 - DELETE → normalmente, el cuerpo viene vacío
 - En caso de error, puede incluir información con más detalle
- Las cabeceras especifican metainformación adicional sobre las respuestas
 - Tipo MIME de los datos devueltos
 - Encoding de la respuesta
 - Lenguaje de la respuesta
 - Antigüedad de la respuesta
 - Longitud de la respuesta
 - Control de cache → si el recurso se puede cachear o no, tiempo de expiración (si se puede cachear), momento de última modificación, ...
 - Control de reintentos
 - Identificación del servidor
 - etc.

CARACTERÍSTICAS SERVICIOS REST

- Se estructuran de forma similar a un sitio de la WWW
- En lugar de páginas HTML, se accederá a la información estructurada (ej. películas, clientes, etc.)
- Está compuesta de muchos sitios web autónomos (como la WWW) → los servicios web REST se consideran autónomos entre sí
- Un servicio REST puede referenciar a otro (links) (como en la WWW) y habrá una serie de servicios generales (ej. caches) que pueden funcionar sobre cualquier servicio

SERVICIOS “STATELESS”

- Cliente – Servidor
 - Clientes invocan directamente URLs para acceder a recursos
- El servidor no guarda información de estado para cada cliente
- Ej. de servicio con estado → FTP
 - El servidor guarda el directorio en el que está cada cliente
 - El resultado de la ejecución de un comando (ej. `get`, para obtener un fichero) depende de ese estado
 - Si el fichero *file.pdf* está en la ruta */docs* y el cliente está en ese directorio el comando *get file.pdf* transferirá el fichero, y en otro caso dará error
- Ej. de servicio sin estado → HTTP
 - Cada petición de un cliente debe contener toda la información necesaria para que el servidor la responda
 - Ej. <http://www.fileserver.com/docs/file.pdf>
- Ventajas de los servicios sin estado
 - Replicación del servicio en múltiples máquinas muy sencilla
 - Basta usar un balanceador de carga que dirige cada petición a una instancia del servicio
 - En un servicio con estado, es necesario garantizar que todas las peticiones de la misma “sesión” van al mismo servidor o usar un servidor de sesiones
 - El servidor no necesita reservar recursos para cada sesión
 - Disminuye los recursos que necesita el servicio
 - Si el cliente falta o se cae en el medio de la sesión, el servidor no tiene que preocuparse de detectarlo ni de liberar recursos
- General → mejora la escalabilidad de los servicios
- Inconveniente → puede ser necesario enviar información adicional con cada petición

RECURSOS Y REPRESENTACIONES

- Una aplicación REST se compone de recursos
- Suelen corresponderse con las entidades persistentes
- 2 tipos de recursos
 - Colección → identifican una serie de recursos del mismo tipo (películas en el ejemplo, clientes en una empresa, ...)
 - Individuales → identifican un elemento concreto (una película, un cliente, ...)
- Cada recurso es identificado mediante un identificador único y global (típicamente URLs)
 - <http://www.movieprovider.com/movies/>
 - Recurso colección que representa todas las películas
 - <http://www.movieprovider.com/movies/3>
 - Una película de movieprovider.com
 - <http://www.acme.com/customers/01235>
 - Un cliente de acme.com
- Los identificadores (URLs) con globales
 - Todo recurso tiene un nombre único a nivel interaplicación
 - Cualquier servicio puede referenciar y acceder a un recurso de cualquier otro servicio
- Al invocar la URL (usando GET), el cliente obtiene una representación del recurso

- La representación debe contener información útil sobre el recurso
 - Recursos colección → lista de los elementos de la colección con información resumen y un enlace para obtener la información completa
 - Recursos individuales → datos del elemento individual (ej. datos de la película, datos del cliente, etc.)
- La representación de un recurso puede variar en el tiempo
 - El identificador está ligado al recurso, no a la representación
 - Ej. si cambian los datos de un cliente de Acme, cambiará la representación
 - La URL apuntará siempre a la representación actual del recurso
- Ej. representación recurso colección

```
<?xml version="1.0" encoding="UTF-8"?>
<customers xmlns="http://www.acme.com/restws/customers">

  <customer>
    <cid>1234</cid>
    <cname>Roadrunner</cname>
    <link href="http://www.acme.com/restws/customers/1234"/>
  </customer>
  <customer>
    <cid>1235</cid>
    <cname>Coyote</cname>
    <link href="http://www.acme.com/restws/customers/1235"/>
  </customer>
  ...
</customers>
```

- Ej. representación recurso individual

```
<?xml version="1.0" encoding="UTF-8"?>
<customer xmlns="http://www.acme.com/restws/customers">
  <cid>1234</cid>
  <cname>Roadrunner</cname>
  <address> Monument Valley, 5 </address>
  <description> He is fast</description>
  <rating> Good </rating>
</customer>
```

INTERFAZ UNIFORME

- Las operaciones disponibles sobre los recursos son siempre las mismas
- Las normas REST no dicen cuáles deben ser esas operaciones, sólo que deben ser siempre las mismas y funcionar igual en todos los servicios
- Los tipos de respuesta (éxito o error) que pueden devolver estas operaciones deben estar también estandarizados
- REST no impone un conjunto de códigos de respuesta concretos, sólo que deben ser los mismos para todos los servicios
- En la práctica, se utiliza HTTP
 - GET
 - Acceso a representaciones
 - Consultas sobre recursos colección
 - <http://www.movieprovider.com/movies?keywords=Dar+Knight>
 - Segura e idempotente
 - PUT
 - Reemplaza la representación de un recurso
 - Si la URL no existe, y el servicio lo permite, crea el recurso
 - No suele permitirse sobre recursos colección
 - No segura y sí idempotente
 - DELETE
 - Borra el recurso
 - No suele permitirse sobre recursos colección
 - No segura y sí idempotente

- POST
 - Sobre recursos colección → suele usarse para crear un nuevo recurso en la colección (ej. añadir una película)
 - El servidor devuelve la URL del nuevo elemento usando una cabecera estándar HTTP
 - Sobre recursos individuales o colección → puede usarse para modelar operaciones no seguras que no encajen con los otros métodos → **overloaded POST**
 - La petición debe incluir en otro lugar (URI, cabeceras o cuerpo) información sobre la operación a invocar sobre el recurso
 - Similar a estilo RPC (no sigue principios de interfaz uniforme)
 - No segura ni idempotente
- Códigos de respuesta estándar HTTP
 - 200 – OK
 - 201 – Created
 - 400 – Bad Request
 - 403 – Forbidden
 - 404 – Not Found
 - 410 – Gone
 - 500 – Internal Error
 - ...
- Códigos de error permanente / temporal
 - Importante poder distinguir entre códigos de respuesta que indican que un error es permanente o temporal
 - La especificación HTTP no dice explícitamente si un código de error tiene asociada una semántica de error permanente o temporal
 - Dentro de una organización siempre es posible establecer convenciones sobre qué códigos representan errores permanentes o temporales
 - Si se quiere poder utilizar intermediarios transparentemente también se debe utilizar las mismas convenciones
 - En la asignatura, atendiendo a su semántica, se utilizarán los códigos de error
 - Errores temporales → 404, 500
 - Errores permanentes → 400, 403, 410
 - NOTA → para elegir el código concreto a utilizar se tendrá en cuenta la semántica del error concreto y si es temporal o permanente
- Cabeceras HTTP estándar para enviar información adicional en la petición y en las respuestas
 - Autenticación
 - Formatos aceptados / enviados
 - Manejo de caches → tiempos de expiración, soporta para peticiones condicionales, etc.
 - ...

SISTEMA EN CAPAS → INTERMEDIARIOS

- Uso IU → permite que haya intermediarios entre cliente y servicio
- Los intermediarios proporcionan funcionalidad adicional sin necesidad de saber nada adicional sobre ellos
- Los intermediarios son transparentes para el cliente y el servicio
 - Es posible porque todos los servicios soportan una interfaz uniforme (operaciones, códigos de respuesta, cabeceras)
- Ej. servidores de cache intermedios
 - En la WWW y en los servicios REST pueden existir múltiples servidores de cache entre cliente y servicio
 - Los servidores de cache sirven una copia del recurso solicitado si la tienen y, si no, invocan al sitio web/servicio real
 - Ejemplos de servidores de cache
 - Los grandes servicios de Internet (ej. Google) pueden usar DNS para redirigir a los clientes a servidores cache de acuerdo a su zona geográfica
 - El administrador de la red de una organización (ej. UDC) puede instalar un proxy que proporcione servicio de caching para optimizar tiempos y ancho de banda

- Las caches funcionan con cualquier servicio y cliente sin necesidad de ninguna preconfiguración ni en el servicio ni en el sistema cache
- La WWW y los servicios REST tienen esta propiedad porque la interfaz uniforme proporciona la información necesaria para el intermediario
 - Petición GET de un recurso no invalida copia en cache
 - Peticiones POST, PUT, DELETE, sí invalidan copia en cache
 - No se cachean peticiones POST, PUT ni DELETE
 - No se deben cachear respuestas que indiquen un código de error temporal (ej. 500 Internal Error), pero sí los que indiquen errores permanentes (ej. 400 Bad Request)
 - Cabeceras de cache proporcionan soporte genérico para indicar tiempo de expiración, recursos que no deben cachearse, peticiones condicionales, etc...
- Otros ejemplos de intermediarios soportados por la WWW y por los servicios REST
 - Proxies → pueden reintentar transparentemente peticiones para proporcionar tolerancia a fallos
 - Necesitan saber si una petición es idempotente o no y si el código de estado de la respuesta es temporal o permanente
 - Traducción de formatos → traducción transparente a un formato de representación no soportado por el servicio
 - Ejs. → cliente espera XML y servicio proporciona JSON
 - Cliente indica en la petición que acepta sólo XML
 - Como el intermediario sabe que es capaz de traducir de JSON a XML, modifica la petición para añadir JSON como formato aceptado
 - El servicio responsable indicando que el formato es JSON
 - El intermediario transforma de JSON a XML y se lo envía al cliente
 - Necesitan una manera estándar de especificar los distintos formatos (tipos MIME), saber qué formatos acepta el cliente (cabeceras Accept) y en qué formato viene la respuesta del servidor (cabecera Content-type)
 - Seguridad
 - Se supone que tenemos un servicio ya hecho y difícil de modificar, que no tiene control de estado
 - Se desea exponer su funcionalidad a otras aplicaciones pero aplicando políticas de control de acceso
 - Es posible añadir un intermediario que permita el acceso sólo a ciertos clientes
 - Necesita que el formato en el que se envía información de autenticación / autorización sea estándar (cabeceras HTTP)

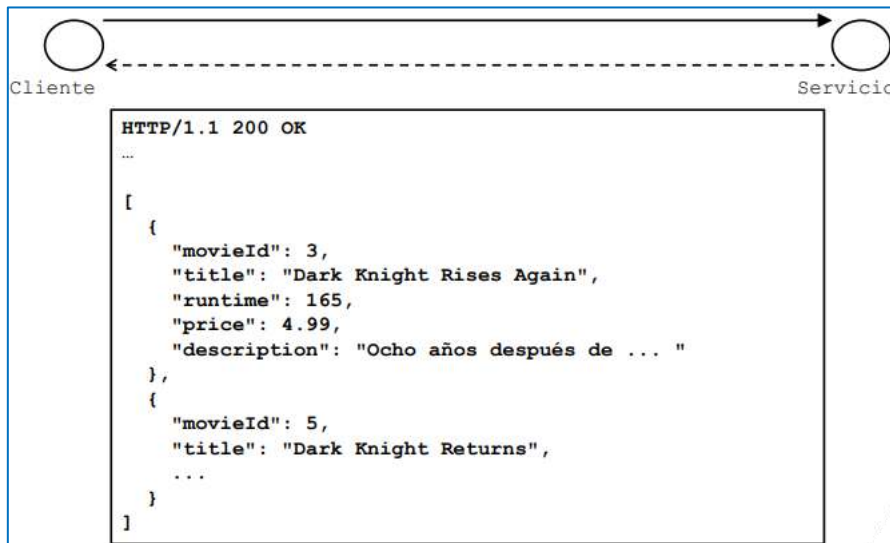
REST EN LA PRÁCTICA

- La mayoría de los servicios reales no utilizan algunos de los principios de REST
 - Uso limitado de hipermedia
 - Uso limitado de representaciones autodescriptivas

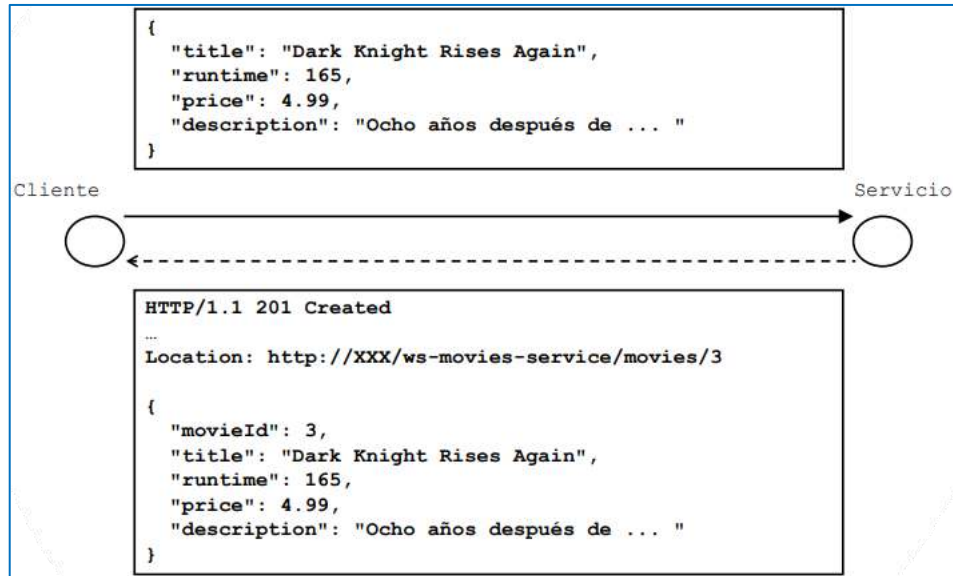
CASO DE ESTUDIO: DISEÑO E IMPLEMENTACIÓN DE UN SERVICIO WEB REST

PROTOCOLO REST

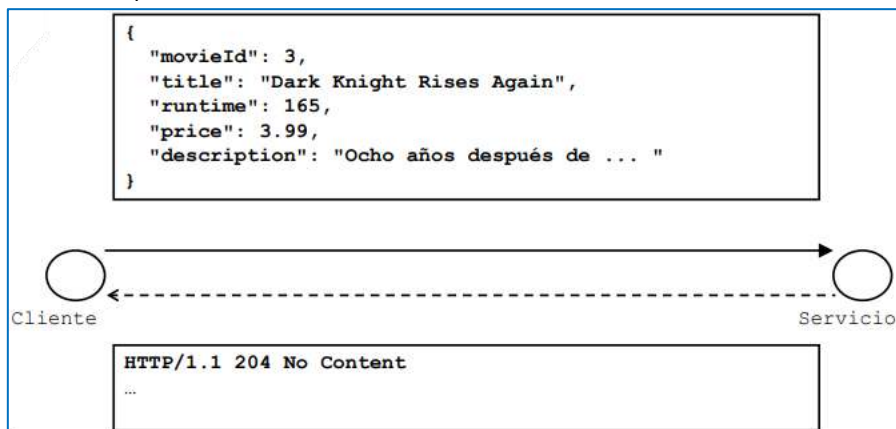
- Se seguirá la misma aproximación que la mayoría de los servicios web REST reales
 - URLs únicas y globales para cada recurso
 - Uso consistente de GET, PUT, POST y DELETE
 - Uso consistente de los códigos de respuesta HTTP
 - Uso de algunas cabeceras estándar HTTP
- Recursos
 - */movies* → Recurso colección
 - GET
 - Lista todas las películas
 - La información de cada película se representa en JSON (en el formato del apartado 5.4)
 - El parámetro *keywords* permite filtrarlas por palabras clave en el título → */movies?keywords=Dark+Knight*
 - Devuelve el código 200 OK
 - POST
 - Añade una nueva película
 - La película se envía en el cuerpo de la petición en formato JSON (sin identificador) (apartado 5.4)
 - Devuelve el código de respuesta HTTP → 201 Created
 - Devuelve la URL de la nueva película usando la cabecera estándar HTTP *Location*
 - El cuerpo de la respuesta devuelve la nueva película creada
- Obtener películas filtradas por palabras clave
 - Petición GET a *http://XXX/ws-movies-service/movies?keywords=Dark+Knight*



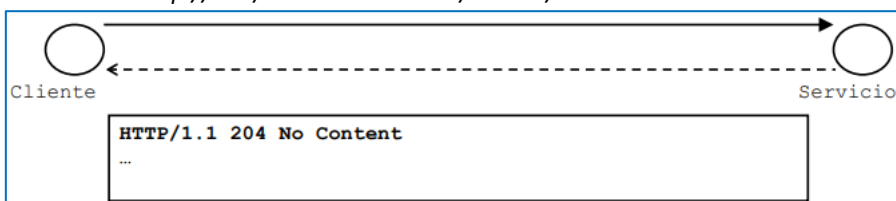
- Añadir la información de una película
 - Petición POST a `http://XXX/ws-movies-service/movies`



- Recursos
 - `/movies/{id}` → Recurso individual por película
 - PUT
 - Modifica la película
 - La película se envía en el cuerpo de la petición en formato JSON (apartado 5.4)
 - El cuerpo de la respuesta va vacío
 - Devuelve el código `204 No Content`
 - DELETE
 - Borra una película
 - El cuerpo de la respuesta va vacío
 - Devuelve el código `204 No Content`
- Actualizar la información de una película
 - Petición PUT a `http://XXX/ws-movies-service/movies/3`



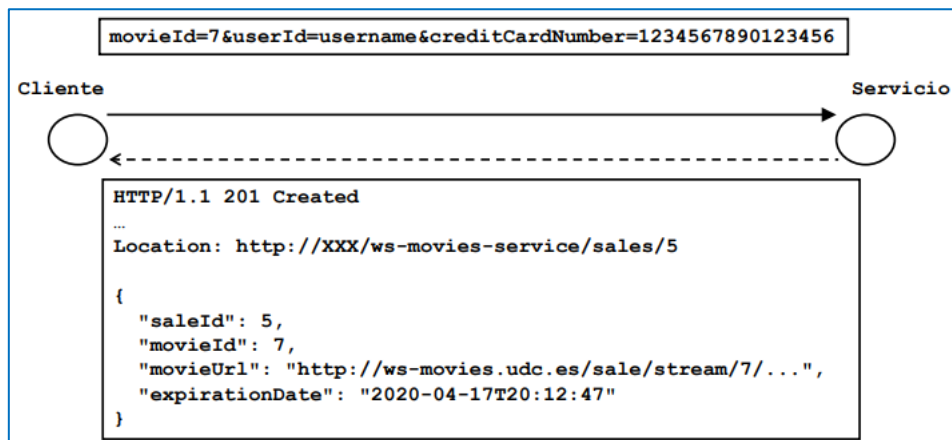
- Eliminar la información de una película
 - Petición DELETE a `http://XXX/ws-movies-service/movies/3`



- Recursos
 - `/sales` → Recurso colección
 - POST
 - Añade una nueva venta (equivalente a comprar película)
 - Los datos de la venta (**userId**, **movieId**, **creditCardNumber**) se reciben como parámetros
 - Devuelve el código de respuesta HTTP → *202 Created*
 - Devuelve la URL de la nueva venta usando la cabecera estándar HTTP *Location*
 - El cuerpo de la respuesta devuelve los datos de la nueva venta creada
 - `/sales/{id}` → Recurso individual por venta
 - GET
 - Obtiene información de la venta
 - La información de cada venta se representa en JSON
 - No es posible borrar ni modificar ventas una vez producidas

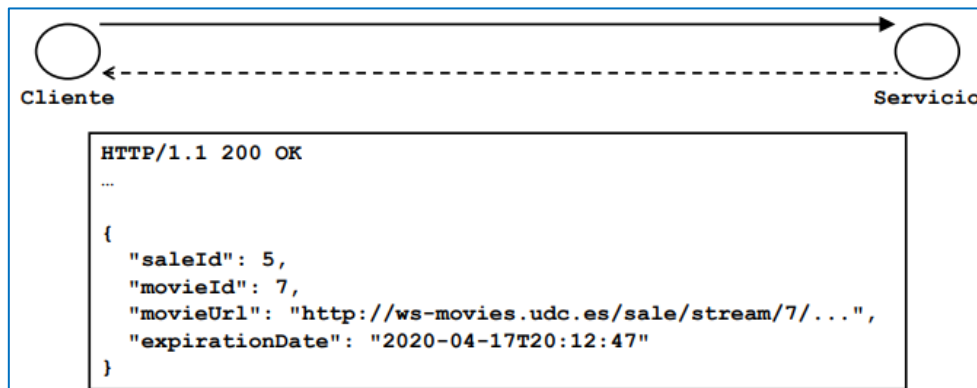
- Añadir la información de una venta

- Petición POST a `http://XXX/ws-movies-service/sales`



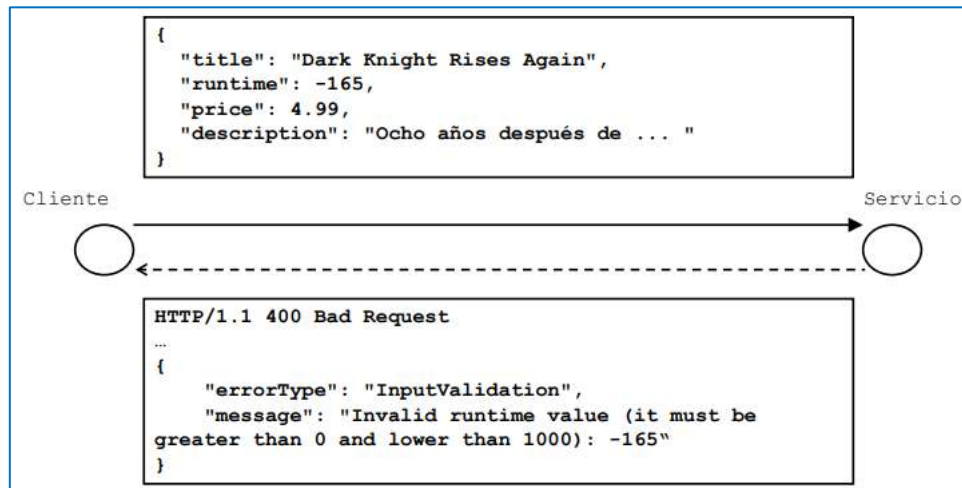
- Obtener la información de una venta

- Petición GET a `http://XXX/ws-movies-service/sales/5`



- Errores → se utilizan los códigos HTTP más próximos a la semántica de la respuesta
 - Parámetros incorrectos → 400 Bad Request
 - Similar a **InputValidationException**
 - Recurso no existe → 404 Not Found
 - Similar a **InstanceNotFoundException**
 - Recurso existió pero ya no existe → 410 Gone
 - Similar a **SaleExpirationException**
 - Se deniega el acceso a la acción solicitada → 403 Forbidden
 - Similar a **MovieNotRemovableException**
 - Error interno de ejecución → 500 Internal Error
- El cuerpo del mensaje puede llevar información adicional (en formato JSON)
 - Normalmente un mismo código de error estará asociado a varias excepciones → debe indicarse de alguna forma a cuál se refiere en cada caso
 - Representación de los datos de la excepción

- Crear una película con datos incorrectos
 - Petición POST a `http://XXX/ws-movies-service/movies`



CONSIDERACIONES DISEÑO REST

- Creación de recursos con POST
 - La URL en la cabecera Location permite al cliente conocer la URL del nuevo recurso creado para referirse a él más tarde
 - Usar una cabecera estándar permite proporcionar semántica para cualquier intermediario y cliente, aunque no conozcan los formatos de nuestro servicio
 - Ej. intermediario que hace transparentemente copia de seguridad de los recursos que se crean
 - Devolver completo el nuevo recurso creado en el cuerpo es útil si creemos que el cliente va a utilizarlo de inmediato (ahorra al cliente una petición HTTP)
 - Si la representación puede ser grande y no es seguro que el cliente la necesite inmediatamente, puede ser mejor enviar sólo el identificador
 - ¿Parámetros o representación ad-hoc en el cuerpo?
 - Usar parámetros es más simple pero está limitado al envío de pares campo/valor
- Códigos de respuesta y de error
 - La ventaja de usar códigos estandarizados es que cualquier cliente o intermediario que utilice ese estándar conoce la semántica de la respuesta
 - Ej. siguiendo las convenciones del apartado 6.2
 - Respuestas 400, 403, 410 son cacheables, y las respuestas 404 y 500 no lo son
 - No tiene sentido reintentar una petición que devuelto 400, 403 o 410, pero sí una que ha devuelto 404 o 500 si la operación es idempotente
 - El cuerpo del mensaje puede llevar información adicional para los clientes específicos del servicio
 - Ej. cuánto hace que expiró la venta
- Modelar otras operaciones que no encajen con operaciones CRUD → Overloaded POST
- Ej. 1 → marcar una película como “destacada”
 - Petición POST a `http://XXX/ws-movies-service/movies/{id}/highlight`
 - POST /url-recurso-individual/operación
 - En {id} debería ir el ID de la película
 - No se puede usar PUT porque es una actualización COMPLETA, y en este caso solo se quiere actualizar un campo
- Ej. 2 → poner de rebajas una película (ej. especificando el porcentaje de rebaja)
 - Petición POST a `http://XXX/ws-movies-service/movies/{id}/reduce`

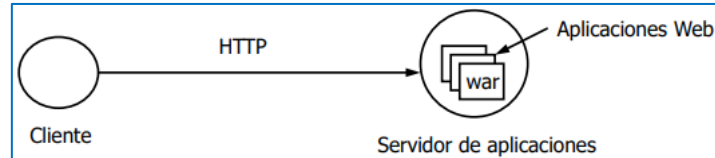
```
percentage=50
```

- Ej. 3 → rebajar el precio de las películas más antiguas que una cierta fecha (ej. especificando el porcentaje de rebaja)
 - Petición POST a `http://XXX/ws-movies-service/movies/reduce`

```
date=01-01-2015&percentage=30
```

APLICACIONES WEB JAVA EE

- En Java EE / Jakarta EE, las **aplicaciones Web** se instalan en **servidores (contenedores) de aplicaciones**

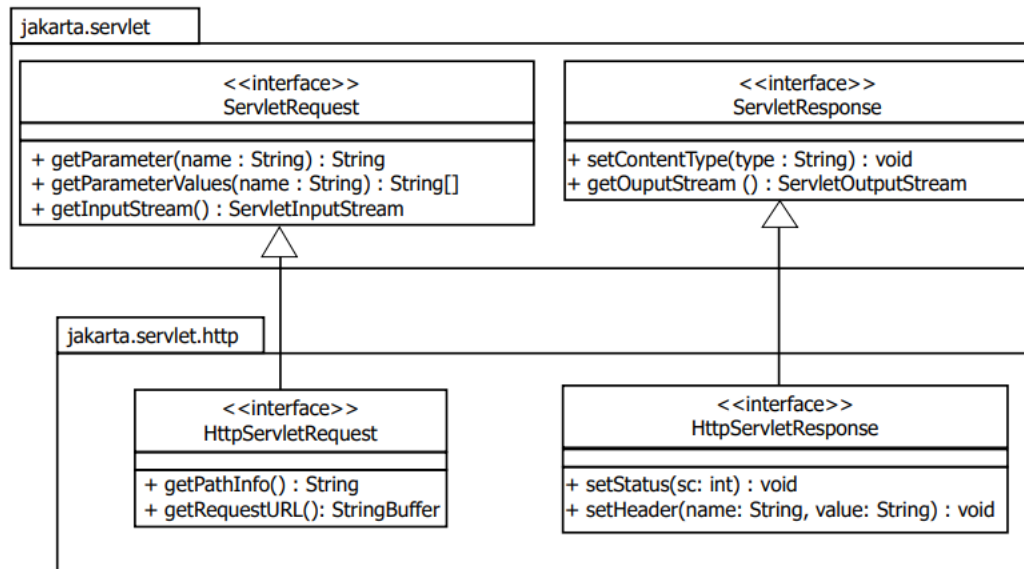


- La **API de programación Web** que ofrece el servidor de aplicaciones está estandarizada
 - Una aplicación web Java EE puede instalarse en cualquier servidor de aplicaciones Java EE
- Las **aplicaciones Web** se distribuyen en **ficheros WAR**
 - Fichero WAR (Web ARchive) → fichero JAR con una estructura de directorios estandarizada
 - Contiene clases objeto (.class), librerías (.jar), ficheros de configuración y otras abstracciones propias de una aplicación Web
- Las APIs de programación Web Java están pensadas para devolver cualquier tipo de información sobre HTTP (no sólo HTML/XHTML)
- Se pueden utilizar para implementar servicios Web
- Se utilizará la API de Servlets para implementar Servicios Web REST

IMPLEMENTACIÓN REST DE LA CAPA SERVICIOS

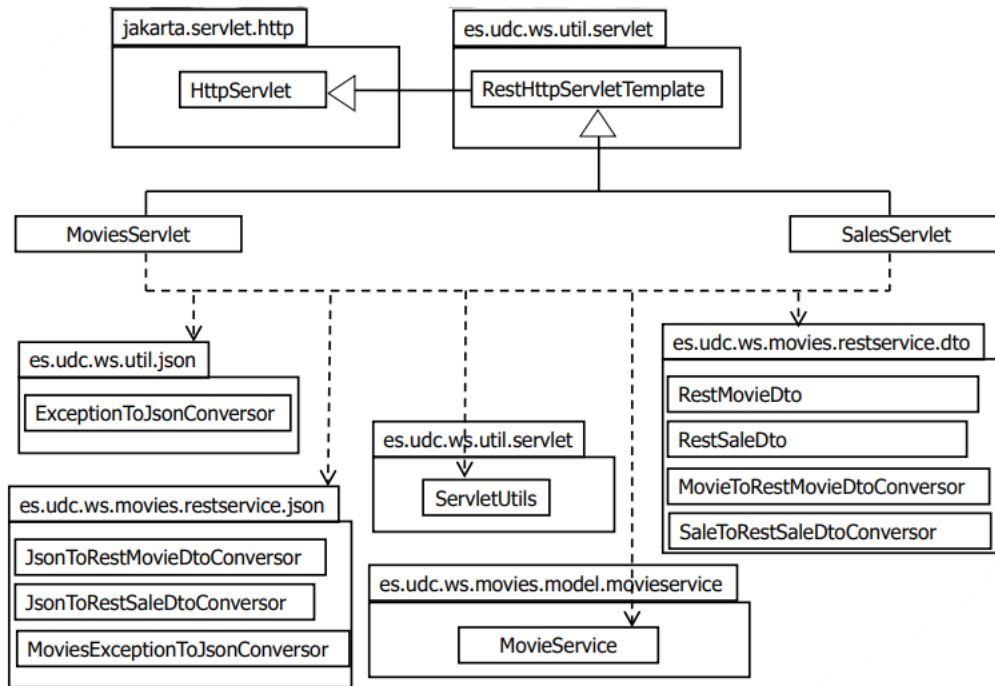
VISIÓN GLOBAL DEL FRAMEWORK DE SERVLETS

- Se utilizará la API de Servlets para la implementación de la capa servicios
- Servlet → clase que
 - Se asocia a una o varias URLs
 - Cuando el servidor recibe una petición sobre esa URL, invoca al servlet asociado
 - El servlet
 - Accede al contenido de la petición
 - Ej. web → valores de un form HTML de búsqueda
 - Ejecuta el código deseado para obtener la respuesta
 - Ej. web → búsqueda en BD por los parámetros del form
 - Codifica la respuesta en el formato deseado
 - Ej. web → página HTML con los resultados de la búsqueda
 - La respuesta devuelta por el servlet será la respuesta enviada por el servidor de aplicaciones al cliente
- El desarrollador implementa un servlet extendiendo de **HttpServlet** y redefine los métodos **doXXX** (ej. **doGet**, **doPost**, **doPut**, **doDelete**, etc.)
- Cuando el servidor de aplicaciones recibe una petición dirigida a un servlet, invoca la operación correspondiente en función del método de la petición (**doGet**, **doPost**, **doPut**, **doDelete**, etc., según la petición HTTP sea GET, POST, PUT, DELETE, etc.)
- Modelo multi-thread de procesamiento de peticiones
 - Cada vez que el servidor de aplicaciones recibe una petición dirigida a un servlet, la petición se procesa en un thread independiente

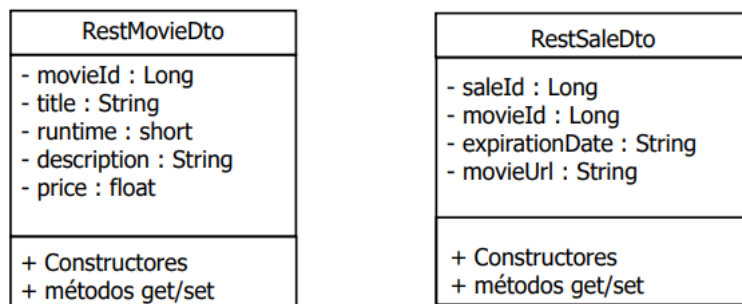


- **ServletRequest**
 - **String getParameter (String name)**
 - Permite obtener el valor de un parámetro univaluado
 - **String[] getParametersValues (String name)**
 - Permite obtener el valor de un parámetro multivaluado
 - Se puede usar con parámetros univaluados
 - **NOTA**
 - Parámetro multivaluado → aquel que tiene (o puede tener) varios valores
 - Al realizar la petición HTTP el parámetro (en la URI o en el cuerpo del mensaje, dependiendo del tipo de petición) se especifica “n” veces, cada una con su valor
 - Ej. `http://example.com/weather?city=Coruna&city=Santiago`
 - **ServletInputStream getInputStream()**
 - Permite leer el cuerpo de la petición
 - **ServletInputStream** es una clase abstracta que extiende **java.io.InputStream**
- **HttpServletRequest**
 - **String getPathInfo()**
 - Devuelve el fragmento del path de la petición a partir de la URL asociada al Servlet (y siempre empieza por '/')
 - Si un Servlet tiene asociado un patrón de URL del tipo **/movies/*** devuelve la parte que encaja con *, precedida de '/'
 - Ejs.
 - `/movies` → **getPathInfo** devuelve null
 - `/movies/` → **getPathInfo** devuelve `"/"`
 - `/movies/123` → **getPathInfo** devuelve `"/123"`
 - `/movies/123/` → **getPathInfo** devuelve `"/123/"`
 - **String getRequestURL ()**
 - Devuelve la URL completa que utilizó el cliente para realizar la petición (sin incluir parámetros)
- **ServletResponse**
 - **void setContentType (String type)**
 - Especifica el tipo de contenido de la respuesta (ej. `"application/json"`)
 - **ServletOutputStream getOutputStream ()**
 - Permite escribir el cuerpo de la respuesta
 - **ServletOutputStream** → clase abstracta que extiende **java.io.OutputStream**
- **HttpServletResponse**
 - **void setStatus (int sc)**
 - Especifica el código de estado de la respuesta
 - **void setHeader (String name, String value)**
 - Añade una cabecera con el nombre y el valor especificados a la respuesta

CAPA SERVICIOS [ES.UDC.WS.MOVIES.RESTSERVICE.SERVLETS]



CAPA SERVICIOS [ES.UDC.WS.MOVIES.RESTSERVICE.DTO]



- **MovieToRestMovieDtoConverter**
 - Realiza conversiones entre **Movie** y **RestMovieDto**
- **SaleToRestSaleDtoConverter**
 - Realiza conversiones entre **Sale** y **RestSaleDto**

CAPA SERVICIOS [ES.UDC.WS.MOVIES.RESTSERVICE.JSON Y ES.UDC.WS.UTIL.JSON]

- **JsonToRestMovieDtoConverter**
 - Realiza conversiones de **RestMovieDto** a/desde JSON
 - Su implementación se vio en la sección 5.4
- **JsonToRestSaleDtoConverter**
 - Transforma a JSON los objetos **RestSaleDto**
- **MoviesExceptionToJsonConverter**
 - Transforma las excepciones del Modelo a su representación JSON
 - Esta representación JSON será un objeto con
 - Un campo llamado **errorType** cuyo valor indica el tipo de excepción/error (ej. **SaleExpiration**)
 - Un campo por cada propiedad de la excepción
- **ExceptionToJsonConverter**
 - Transforma las excepciones del módulo de utilidades a su representación JSON siguiendo el mismo esquema que **MoviesExceptionToJsonConverter**

CAPA SERVICIOS [ES.UDC.WS.UTIL.SERVLET]

- **ServletUtils**

- Clase con métodos utilidad para implementar Servlets

ServletUtils
<pre> + writeServiceResponse(response : HttpServletResponse, responseCode:int, rootNode : JsonNode, headers : Map<String,String>) : void + normalizePath(url : String) : String + getMandatoryParameter(req : HttpServletRequest, paramName : String) : String + getMandatoryParameterAsLong(req : HttpServletRequest, paramName : String) : Long + checkEmptyPath(req : HttpServletRequest) : void + getIdFromPath(req : HttpServletRequest, resourceName : String) : Long </pre>

- **writeServiceResponse** escribe una respuesta HTTP y recibe como parámetros
 - Un objeto de tipo **HttpServletResponse**
 - El código a enviar en la respuesta
 - El cuerpo a enviar en la respuesta
 - Objeto **JsonNode** de Jackson que debe ser el nodo raíz del árbol cuyo JSON se quiere enviar
 - Un mapa con los nombres y valores de cabeceras a añadir a la respuesta
- **normalizePath** → recibe un String y devuelve otro String igual al recibido quitándole el carácter '/' en caso de que finalice con él
- **getMandatoryParameter** → obtiene el valor del parámetro indicado o lanza una **InputValidationException** si el parámetro no viene en la petición
- **getMandatoryParameterAsLong** → hace lo mismo que **getMandatoryParameter** pero convierte el valor del parámetro a Long y lanza una **InputValidationException** si no es posible
- **checkEmptyPath** → comprueba si el path de una petición HTTP es vacío, nulo o está compuesto solo por "/" y en caso negativo lanza una **InputValidationException**
- **getIdFromPath** → obtiene un identificador de tipo Long a partir del path de una petición HTTP que siga el formato "/<id>"
 - En caso de no poder obtener el identificador lanza una **InputValidationException**

ES.UDC.WS.UTIL.SERVLET.RESTHTTPSERVLETTEMPLATE [CAPA SERVICIOS]

```

public class RestHttpServletTemplate extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
        try {
            processGet(req, resp);

        } catch (InstanceNotFoundException ex) {
            ServletUtils.writeServiceResponse(resp,
                HttpServletResponse.SC_NOT_FOUND,
                ExceptionToJsonConversor.toInstanceNotFoundException(ex), null);
        } catch (InputValidationException ex) {
            ServletUtils.writeServiceResponse(resp,
                HttpServletResponse.SC_BAD_REQUEST,
                ExceptionToJsonConversor.toInputValidationException(ex), null);
        } catch (ParsingException ex) {
            ServletUtils.writeServiceResponse(resp,
                HttpServletResponse.SC_BAD_REQUEST,
                ExceptionToJsonConversor.toInputValidationException(
                    new InputValidationException(ex.getMessage()), null);
        }
    }

    protected void processGet(HttpServletRequest req,
        HttpServletResponse resp) throws IOException,
        InstanceNotFoundException, InputValidationException {

        ServletUtils.writeServiceResponse(resp,
            HttpServletResponse.SC_NOT_IMPLEMENTED, null, null);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {...}

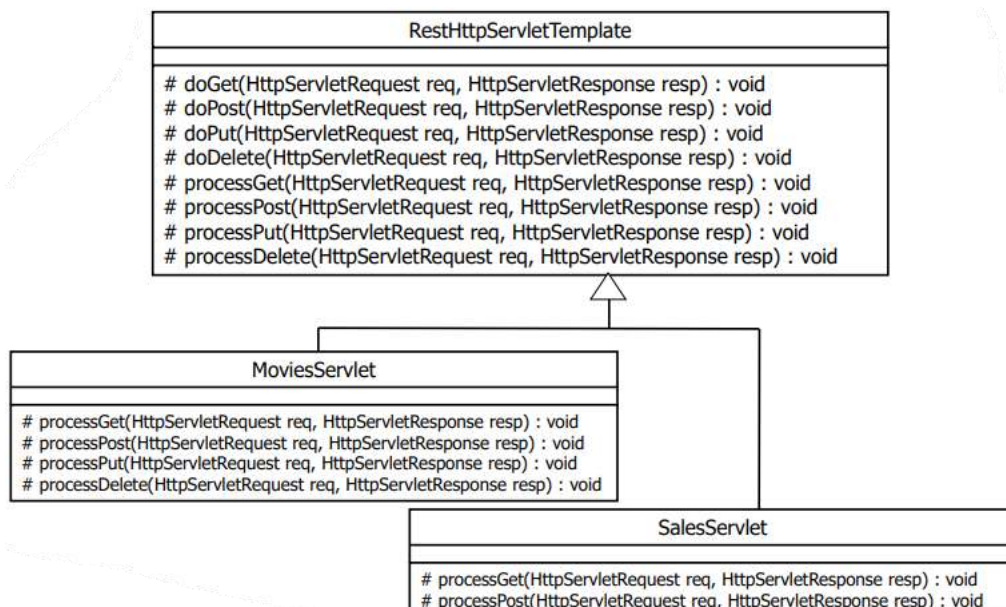
    protected void processPost(HttpServletRequest req,
        HttpServletResponse resp) throws IOException,
        InstanceNotFoundException, InputValidationException {...}

    // doPut + processPut
    ...

    // doDelete + processDelete
    ...
}

```

- RestHttpServletTemplate



COMENTARIOS

- **RestHttpServletTemplate**

- Clase para facilitar la implementación de Servlets que invoquen a operaciones de una capa Modelo que lance excepciones definidas en **ws-util**
- Extiende a **HttpServlet**
- Patrón Template Method → implementa las operaciones **doXXX** (Get, Post, Put y Delete) llamando a la operación **processXXX**
 - Se captura las excepciones del módulo **ws-utils** al procesar cualquier petición y se envía la respuesta adecuada para cada una de ellas → los Servlets que extiendan de esta clase no tienen que preocuparse de tratar estas excepciones
- La implementación por defecto de **processXXX** devuelve un código de respuesta 501 (**SC_NOT_IMPLEMENTED**) para indicar que esa operación no está implementada
 - Los Servlets que extiendan de esta clase solo tienen que implementar los métodos processXXX acordes a los métodos que deban soportar

ES.UDC.WS.MOVIES.RESTSERVICE.SERVLET.MOVIESSERVLET [CAPA SERVICIOS]

```
public class MoviesServlet extends RestHttpServletTemplate {

    @Override
    protected void processGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException, InputValidationException {

        ServletUtils.checkEmptyPath(req);
        String keyWords = req.getParameter("keywords");

        List<Movie> movies =
            MovieServiceFactory.getService().findMovies(keyWords);

        List<RestMovieDto> movieDtos =
            MovieToRestMovieDtoConversor.toRestMovieDtos(movies);

        ServletUtils.writeServiceResponse(resp,
            HttpServletResponse.SC_OK,
            JsonToRestMovieDtoConversor.toArrayNode(movieDtos),
            null);
    }

    @Override
    protected void processPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException, InputValidationException {

        ServletUtils.checkEmptyPath(req);

        RestMovieDto movieDto =
            JsonToRestMovieDtoConversor.toRestMovieDto(req.getInputStream());
        Movie movie = MovieToRestMovieDtoConversor.toMovie(movieDto);

        movie = MovieServiceFactory.getService().addMovie(movie);

        movieDto = MovieToRestMovieDtoConversor.toRestMovieDto(movie);
        String movieURL =
            ServletUtils.normalizePath(req.getRequestURL().toString()) + "/" +
            movie.getMovieId();
        Map<String, String> headers = new HashMap<>(1);
        headers.put("Location", movieURL);

        ServletUtils.writeServiceResponse(resp,
            HttpServletResponse.SC_CREATED,
            JsonToRestMovieDtoConversor.toObjectNode(movieDto),
            headers);
    }
}
```



```

@Override
protected void processDelete(HttpServletRequest req, HttpServletResponse resp)
    throws IOException, InputValidationException, InstanceNotFoundException {

    Long movieId = ServletUtils.getIdFromPath(req, "movie");

    try {
        MovieServiceFactory.getService().removeMovie(movieId);
    } catch (MovieNotRemovableException ex) {
        ServletUtils.writeServiceResponse(resp,
            HttpServletResponse.SC_FORBIDDEN,
            MoviesExceptionToJsonConversor.toMovieNotRemovableException(ex),
            null);
        return;
    }

    ServletUtils.writeServiceResponse(resp,
        HttpServletResponse.SC_NO_CONTENT, null, null);
}

```

```

@Override
protected void processPut(HttpServletRequest req, HttpServletResponse resp)
    throws IOException, InputValidationException,
    InstanceNotFoundException {

    Long movieId = ServletUtils.getIdFromPath(req, "movie");

    RestMovieDto movieDto =
        JsonToRestMovieDtoConversor.toRestMovieDto(req.getInputStream());
    if (!movieId.equals(movieDto.getMovieId())) {
        throw new InputValidationException("Invalid Request: invalid movieId");
    }
    Movie movie = MovieToRestMovieDtoConversor.toMovie(movieDto);

    MovieServiceFactory.getService().updateMovie(movie);

    ServletUtils.writeServiceResponse(resp,
        HttpServletResponse.SC_NO_CONTENT, null, null);
}
}

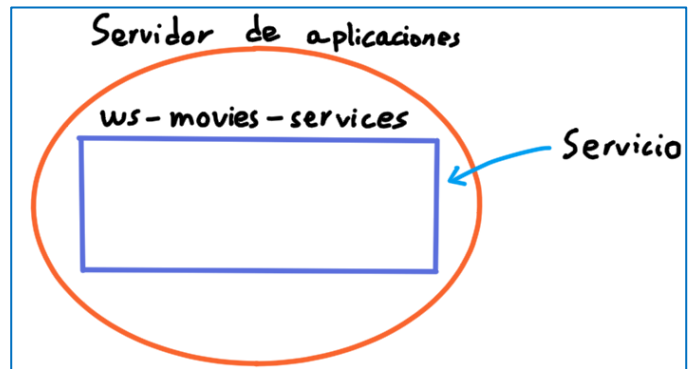
```

COMENTARIOS

- **SalesServlet**
 - **processPost** → similar al de **MoviesServlet** (invocando la operación **buyMovie** del modelo)
 - El cliente envía los datos de entrada como parámetros en el cuerpo de la petición en lugar de usar una representación JSON (en un POST los parámetros van en el cuerpo y no en la URL)
 - Los parámetros se leen de la misma forma que si fuese una petición GET y los parámetros figurasen en la URL (usando los métodos **getParameter** o **getParameterValues** de **HttpServletRequest**)
 - **processGet** → obtiene el identificador de la venta del path de la petición (de forma similar a **processDelete** de **MoviesServlet**)
- Los servlet no tratan **RuntimeException**
 - Representa errores relativos a la infraestructura usada
 - El servidor de aplicaciones captura las excepciones de runtime, y si se producen, devuelve una respuesta con código de estado 500 (INTERNAL SERVER ERROR) (es lo que se desea)

EMPAQUETAMIENTO DE UNA APLICACIÓN WEB

- **jar cvf aplicacionWeb.war directorio**
 - Opciones similares al comando Unix **tar**
 - El nombre de una aplicación Web no tiene porque coincidir con el de su fichero **.war**
 - El nombre se decide al instalar el fichero **.war** en el servidor de aplicaciones
- Maven genera automáticamente ficheros **.war** para los módulos con empaquetamiento "war"
- Estructura de un fichero **.war**
 - Directorio **WEB-INF/classes**
 - Ficheros **.class** que conforman la aplicación Web, agrupados en directorios según su estructura en paquetes
 - Sin ficheros fuente
 - Directorio **WEB-INF/lib**
 - Ficheros **.jar** de librerías que usa la aplicación
 - Sin ficheros fuente
 - **WEB-INF/web.xml**
 - Configuración estándar de la aplicación Web
- Un fichero **.war** se puede instalar (deployment) en cualquier otro servidor de aplicaciones Java EE / Jakarta EE



WEB.XML [CAPA SERVICIOS]

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.4"
  xmlns="http://java.sun.com/xml/ns/j2ee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
    http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
  <display-name>JavaExamples Movies Service</display-name>

  <!-- REST service -->
  <servlet>
    <display-name>MoviesServlet</display-name>
    <servlet-name>MoviesServlet</servlet-name>
    <servlet-class>
      es.udc.ws.movies.restservice.servlets.MoviesServlet
    </servlet-class>
  </servlet>

  <servlet>
    <display-name>SalesServlet</display-name>
    <servlet-name>SalesServlet</servlet-name>
    <servlet-class>
      es.udc.ws.movies.restservice.servlets.SalesServlet
    </servlet-class>
  </servlet>

  <servlet-mapping> — Se le dice al servidor de aplicaciones cuales son las peticiones que tiene que procesar
    <servlet-name>MoviesServlet</servlet-name>
    <url-pattern>/movies/*</url-pattern>
  </servlet-mapping>
  <servlet-mapping>
    <servlet-name>SalesServlet</servlet-name>
    <url-pattern>/sales/*</url-pattern>
  </servlet-mapping>

  ...
</web-app>
```

en este patrón no cuenta la dirección del servicio, es decir /movies...

se pone así por si se quiere cambiar de puerto, o servidor, para no tener que modificar todo

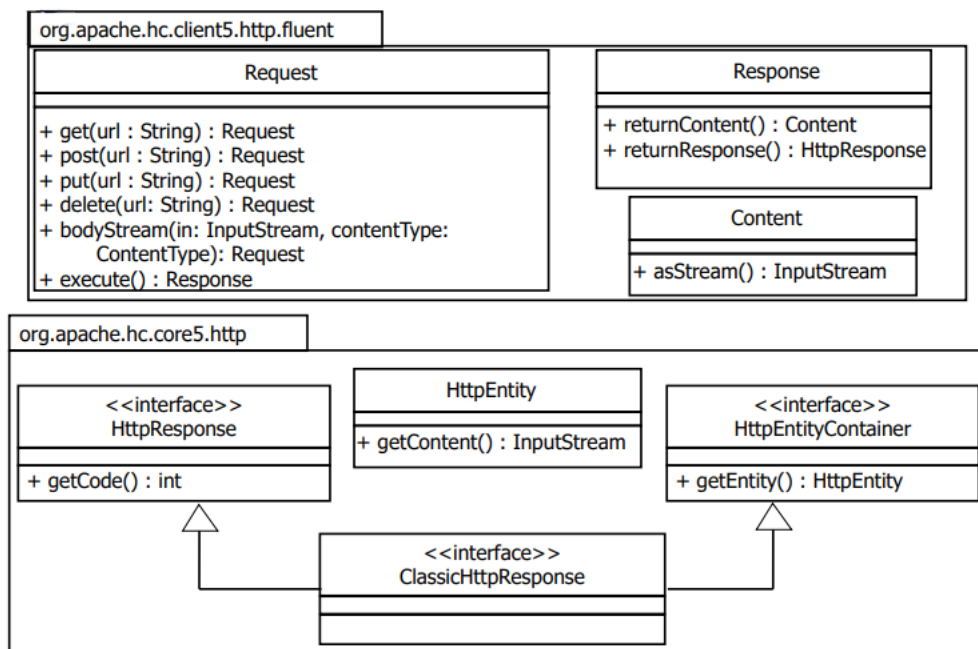
COMENTARIOS

- **display-name**
 - Nombre visual que mostrará la aplicación de administración del servidor de aplicaciones para esta aplicación Web
- **servlet**
 - Permite definir un servlet, especificando un nombre visual (**display-name**), un nombre (**servlet-name**) para referirse al servlet desde el resto del fichero **web.xml** y el nombre completo de la clase que lo implementa (**servlet-class**)
- **servlet-mapping**
 - Permite especificar las URLs a las que responderá el servlet especificado en **servlet-name**
 - **url-pattern** permite especificar una URL concreta o un patrón (ej. **/movies/***)
 - NOTA → las URLs especificadas no incluyen la parte inicial (**http://direcciónServidor[:puerto]/nombreAplicaciónWeb**)
 - Si el servicio cambia de dirección y/o puerto, o la aplicación Web se reinstala con otro nombre, no hay que cambiar el fichero **web.xml**

IMPLEMENTACIÓN REST DE LA CAPA ACCESO A SERVICIOS

VISIÓN GLOBAL DE HTTPCLIENT

- Se usará HttpClient para la implementación de la capa Acceso a Servicios
- Framework open-source de Apache
- Forma parte de Apache HttpComponents (<http://hc.apache.org>)
- Se utilizará la “Fluent API”
 - Simplifica el uso de HttpClient, permitiendo que el desarrollador no tenga que ocuparse de aspectos como el manejo y liberación de conexiones
 - Utiliza la idea de “Method chaining”
 - No permite utilizar toda la funcionalidad de HttpClient, solo la más común



- **Request**

- Representa una petición HTTP
- Por cada tipo de operación HTTP (GET, POST, etc.) existe un método del mismo nombre que recibe la URL del recurso
- El método **bodyStream** permite asignar un contenido cualquiera al cuerpo de la petición
- El método **execute** envía la petición y devuelve un objeto **Response** que representa la respuesta a la petición
 - Permite acceder al contenido del cuerpo pero no al código de respuesta
 - El método **returnResponse** de **Response** permite obtener un objeto que implementa **HttpResponse** → esta interfaz permite acceder al código de respuesta pero no al contenido del cuerpo
 - Se hará un cast a la interfaz **ClassicHttpResponse** para poder acceder al código de respuesta y al contenido del cuerpo
- Ej.

```
ClassicHttpResponse response = (ClassicHttpResponse)
    Request.post("http://...").
        bodyStream(toInputStream(someJsonContent),
            ContentType.create("application/json")).
        execute().returnResponse();
```

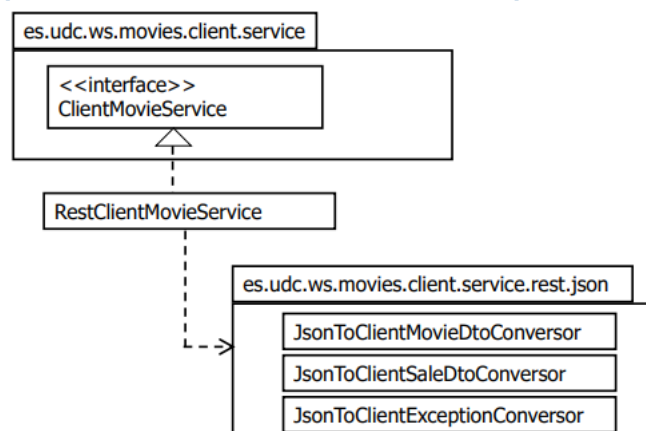
- En peticiones GET los parámetros se envían en la URL
 - En peticiones POST, van en el cuerpo de la petición en un formato establecido (el mismo utilizado para enviar datos de formularios HTML)
 - El método **bodyForm** permite incluir en el cuerpo de la petición parámetros en este formato
- ```
ClassicHttpResponse response = (ClassicHttpResponse)
 Request.post("http://...").
 bodyForm(Form.form().add("param1", "value1").add(
 "param2", "value2").build()).
 execute().returnResponse();
```

- **ClassicHttpResponse**

- Representa la respuesta a una petición HTTP
- El código de respuesta puede obtenerse llamando a **getStatusCode()**
  - La clase **HttpStatus** declara constantes para cada uno de los códigos de estado → **SC\_OK** (200), **SC\_NOT\_FOUND** (404), **SC\_INTERNAL\_SERVER\_ERROR** (500), etc.
- El cuerpo de la respuesta puede obtenerse llamando a **getEntity().getContent()**
- El método **getFirstHeader(name)** permite obtener un objeto **Header** representando la cabecera con el nombre especificado
- El método **getValue()** de **Header** permite obtener su valor

---

## CAPA ACCESO A SERVICIOS [ES.UDC.WS.MOVIES.CLIENT.SERVICE.REST]

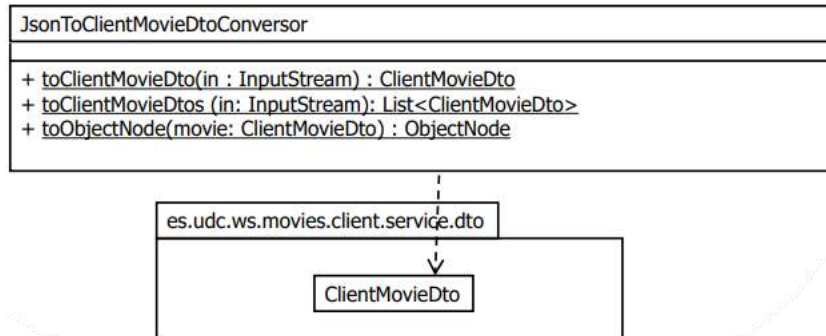


- **RestClientMovieService**

- Utiliza Jakarta Commons HttpClient para realizar las peticiones HTTP
- Utiliza las clases del subpaquete **json** para parsear / generar JSON

## CAPA ACCESO A SERVICIOS [ES.UDC.WS.CLIENT.SERVICE.REST.MOVIES.JSON]

- **JsonToClientMovieDtoConversor**
  - Realiza conversiones de **ClientMovieDto** a/desde JSON
  - **toClientMovieDto** y **toObjectNode**
    - Implementación similar a la de los métodos equivalentes en **JsonToRestMovieDtoConversor** (apartado 5.4), pero usando **ClientMovieDto** en lugar de **RestMovieDto**
  - **toClientMovieDtos**
    - Recibe un stream conteniendo una lista de películas en JSON
    - Se utiliza para parsear las películas resultado de una consulta



- **JsonToClientSaleDtoConversor**
  - Obtiene los datos de una venta (**ClientSaleDto**) desde su representación en JSON
  - No se necesitan métodos que conviertan una venta a su representación JSON, ni que conviertan listas de ventas
- **JsonToClientExceptionConversor**
  - Convierte desde la representación en JSON de las excepciones a las excepciones usadas por el cliente

## ES.UDC.WS.MOVIES.CLIENT.SERVICE.REST.RESCLIENTMOVIESERVICE [CAPA ACCESO A SERVICIOS]

```

@Override
public Long addMovie(ClientMovieDto movie) throws InputValidationException {
 try {
 ClassicHttpResponse response = (ClassicHttpResponse)
 Request.post(getEndpointAddress() + "movies").
 bodyStream(toInputStream(movie),
 ContentType.create("application/json")).
 execute().returnResponse();

 validateStatusCode(HttpStatus.SC_CREATED, response);

 return JsonToClientMovieDtoConversor.toClientMovieDto(
 response.getEntity().getContent().getMovieId());
 } catch (InputValidationException e) {
 throw e;
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}

```

```

@Override
public List<ClientMovieDto> findMovies(String keywords) {
 try {
 ClassicHttpResponse response = (ClassicHttpResponse)
 Request.get(getEndpointAddress() + "movies?keywords="
 + URLEncoder.encode(keywords, "UTF-8")).
 execute().returnResponse();

 validateStatusCode(HttpStatus.SC_OK, response);

 return JsonToClientMovieDtoConversor.toClientMovieDtos(
 response.getEntity().getContent());
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}

```

```

@Override
public void removeMovie(Long movieId) throws InstanceNotFoundException,
 ClientMovieNotRemovableException {

 try {
 ClassicHttpResponse response = (ClassicHttpResponse)
 Request.delete(getEndpointAddress() + "movies/" + movieId).
 execute().returnResponse();

 validateStatusCode(HttpStatus.SC_NO_CONTENT, response);

 } catch (InstanceNotFoundException |
 ClientMovieNotRemovableException e) {
 throw e;
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}

```

```

@Override
public Long buyMovie(Long movieId, String userId, String creditCardNumber)
 throws InstanceNotFoundException, InputValidationException {

 try {
 ClassicHttpResponse response = (ClassicHttpResponse)
 Request.post(getEndpointAddress() + "sales").
 bodyForm(Form.form().
 add("movieId", Long.toString(movieId)).
 add("userId", userId).
 add("creditCardNumber", creditCardNumber).
 build()).
 execute().returnResponse();

 validateStatusCode(HttpStatus.SC_CREATED, response);

 return JsonToClientSaleDtoConversor.toClientSaleDto(
 response.getEntity().getContent()).getSaleId();

 } catch (InputValidationException | InstanceNotFoundException e) {
 throw e;
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}

```

```

private InputStream toInputStream(ClientMovieDto movie) {
 try {
 ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
 ObjectMapper objectMapper = ObjectMapperFactory.instance();

 objectMapper.
 writer(new DefaultPrettyPrinter()).
 writeValue(outputStream,
 JsonToClientMovieDtoConversor.toObjectNode(movie));

 return new ByteArrayInputStream(outputStream.toByteArray());

 } catch (IOException e) {
 throw new RuntimeException(e);
 }
}

```

```

private void validateStatusCode(int successCode, ClassicHttpResponse
 response) throws Exception {
 try {
 int statusCode = response.getStatusCode();
 /* Success? */
 if (statusCode == successCode) { return; }

 /* Handler error. */
 switch (statusCode) {
 case HttpStatus.SC_NOT_FOUND -> throw
 JsonToClientExceptionConversor.fromNotFoundErrorCode(
 response.getEntity().getContent());
 case HttpStatus.SC_BAD_REQUEST -> throw
 JsonToClientExceptionConversor.fromBadRequestErrorCode(
 response.getEntity().getContent());
 case HttpStatus.SC_FORBIDDEN -> throw
 JsonToClientExceptionConversor.fromForbiddenErrorCode(
 response.getEntity().getContent());
 case HttpStatus.SC_GONE -> throw
 JsonToClientExceptionConversor.fromGoneErrorCode(
 response.getEntity().getContent());
 default -> throw new RuntimeException(
 "HTTP error; status code = " + statusCode);
 }
 } catch (IOException e) {
 throw new RuntimeException(e);
 }
}

```

## COMENTARIOS FINALES

- **toInputStream** convierte un objeto **ClientMovieDto** a su representación JSON y devuelve un **InputStream** del que poder leer el JSON
- No se muestran los métodos
  - **updateMovie** → invoca un PUT sobre el recurso `/movies/{id}` enviando los datos de la película en el cuerpo (similar a **addMovie**)
  - **getMovieUrl** → invoca un GET sobre el recurso `/sales/{id}` y devuelve la URL del objeto **ClientSaleDto** obtenido como respuesta
  - **getEndpointAddress** → obtiene la parte fija de la URL desde el fichero de configuración usando la clase **ConfigurationParametersManager**
- **validateStatusCode**
  - En el caso de estudio el código de error HTTP permitiría identificar la excepción
  - En general, debe usarse el cuerpo del mensaje ya que podría haber varias excepciones asociadas al mismo código HTTP (implementaciones de los métodos **JsonToClientExceptionConverter.fromXXXErrorCode**)

## VALIDACIÓN DE DOCUMENTOS

- Tanto los clientes como el servicio del ejemplo comprueban que el JSON está bien formado (lo hace Jackson automáticamente)
- Ni los clientes ni el servicio comprueban que el documento es válido
  - **Pero se hacen las comprobaciones necesarias**
  - La capa modelo comprueba que los parámetros y datos recibidos son válidos (ej. números de tarjeta de crédito)
- Ventajas de no realizar una validación estricta
  - Eficiencia
    - Especialmente importante para la implementación de servicios (que potencialmente pueden recibir muchas peticiones concurrentes)
  - Evolución en el tiempo
    - Es posible extender el JSON/XML del protocolo sin que dejen de funcionar clientes y/o servicios
    - **NOTA** → con JSON Schema sería posible hacer una validación no estricta (validar solamente un conjunto fijo de campos y no dar error si el documento contiene más campos)
      - Puede conseguirse no usando **additionalProperties** en los objetos (o especificando **true** como valor)
      - Estas características, que no tienen los esquemas XML, ayudan a facilitar la evolución en el tiempo
- Ej. 1 (evolución en el tiempo)
  - Muchas empresas han construido clientes de búsqueda de películas
  - Más adelante la proveedora del servicio decide añadir el tag **rating** (puntuación) a la información de una película
  - Modifica la implementación del servicio y el esquema JSON/XML → en este momento, los clientes no están actualizados
  - Sin embargo, no dejan de funcionar, dado que en el JSON/XML que reciben sólo preguntan por los tags que conocen (ej. **movieId**, **title**, **runtime**, etc.)
  - Si los clientes tuviesen una copia local del viejo esquema e intentasen validar contra ella, fallaría la validación hasta que se actualizase el esquema
    - Podría descargarse es esquema cada vez, pero es ineficiente
- Ej. 2 (evolución en el tiempo)
  - Los clientes que introdujesen datos de nuevas películas sin el nuevo tag también podrían seguir funcionando
  - No enviarían la puntuación de la película
  - El servicio tendría que tratar el tag **rating** como opcional (podría validarse siempre que el esquema especificase este elemento como opcional)